

Руководство по использованию новой системы стихосложения

Что за система стихосложения?

Система стихосложения – это, по сути, мини-игра про сочинение стихов, в которую вы играете в DDLC в конце каждого дня, где вы пишете «стих» для одного из членов клуба.

Что изменилось?

Много чего. В основном, использование словарей, классов и наследования для слов, персонажей, которым можно посвятить стих, и многое другое.

Как мне использовать новую систему?

На данный момент система использует множество модулей:

PoemWord

Этот класс определяет слова для стихотворения, которые либо достаются из файла *poem_game/poemwords.txt*, либо добавляются вручную для некоторых ключевых слов, как, например, в актах 2-3. Эта система осталась такой же, как и в DDLC.

ChibiTrans

ChibiTrans отвечает за перемещение девочек на тетради и подсказывание, когда выбирается их слово. Это полезно для добавления в игру персонажа, у которого нет счётчика привязанности (как у Моники в третьем акте).

Chibi(name)

Chibi отвечает за хранение данных привязанности во время мини-игры. Он наследует ChibiTrans, а это значит, что он также содержит информацию о прыжках и данные об очках. Это полезно для добавления персонажа, у которого есть счётчик привязанности (Сайори, Нацуки, Юри). Принимает на вход строку с именем.

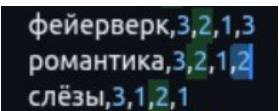
Как мне вызывать новую мини-игру?

Как и в DDLC, вам нужно прописать *call poem*

Как мне добавить нового персонажа?

В этом разделе предполагается, что вы не меняли кардинально исходный код или экран мини-игры.

1. Добавьте ещё одну цифру через запятую к словам в файле *poem_game/poemwords.txt*, чтобы указать, как сильно вашему персонажу нравятся эти слова. (добавьте ещё слова при необходимости).

а. 

2. Добавьте ещё одну переменную в класс PoemWord для этого персонажа и добавьте self-переменную в функции __init__:

```
class PoemWord:
    def __init__(self, s, n, y, l, glitch=False):
        self.sPoint = s
        self.nPoint = n
        self.yPoint = y
        self.lPoint = l
        self.glitch = glitch
```

а.

3. Создайте переменную с наследованием класса *Chibi* или *ChibiTrans*.

- а. Не забывайте, что *Chibi* предназначен для хранения данных и перемещения, в то время как *ChibiTrans* отвечает только за перемещение.

```
chibi_m = ChibiTrans()
chibi_y = Chibi('yuri')
chibi_l = Chibi('libitina')
chibi_a = ChibiTrans()
```

и.

4. Измените строки 153-177, чтобы добавить изображение подпрыгивающего персонажа в первом и втором актах (если применимо):

```
# Акт 1
if persistent.playthrough == 0:
    if t.sPoint >= 3:
        renpy.show("s_sticker hop")
    if t.nPoint >= 3:
        renpy.show("n_sticker hop")
    if t.yPoint >= 3:
        renpy.show("y_sticker hop")
    if t.lPoint >= 3:
        renpy.show("l_sticker hop")
else:
    # Акт 2
    if persistent.playthrough == 2 and chapter == 2 and random.randint(0,10) == 0: renpy.show("m_sticker hop") #
    elif t.nPoint > t.yPoint: renpy.show("n_sticker hop") # Поскольку во Втором акте остались Юри и Нацуки, та
    подпрыгнет.
    elif persistent.playthrough == 2 and not persistent.seen_sticker and random.randint(0,100) == 0:
        renpy.show("y_sticker hopg") # "y_sticker_2g.png". Если мы его ещё не видели, есть шанс 1/100 на то, чт
        persistent.seen_sticker = True
    elif persistent.playthrough == 2 and chapter == 2: renpy.show("y_sticker_cut hop") # Стикер Юри с порезанн
    else: renpy.show("y_sticker hop")
```

а.

5. Добавьте строку над *progress += 1*, похожую на следующее:

```
# Добавляет очки персонажам и увеличивает прогресс на 1.
chibi_s.charPointTotal += t.sPoint
chibi_n.charPointTotal += t.nPoint
chibi_y.charPointTotal += t.yPoint
chibi_l.charPointTotal += t.lPoint
progress += 1
```

а.

6. Перед функцией *exes* внутри *poem_game_finish*, добавьте строку, похожую на следующее:

```
y_poemappeal[chapter] = chibi_y.calculate_appeal()
l_poemappeal[chapter] = chibi_l.calculate_appeal()

# У победительницы значение признания всегда будет равно 1 (т.е. стих ей понравился).
exes(f"{poemwinner[chapter][0]}_poemappeal[chapter] = 1")
```

а.

- i. Добавьте также определение в *definitions/definitions.rpy*:

```
default y_poemappeal = [0, 0, 0]
default m_poemappeal = [0, 0, 0]
default l_poemappeal = [0, 0, 0]
```

1.

7. Добавьте стоячее изображение своего персонажа в лейбле роет таким образом:

- a. При необходимости вы можете изменить применяемую трансформацию.

```
if persistent.playthrough == 2 and chapter == 2:
    show y_sticker_cut at sticker_right # Заменяет чиби Юри на оный с "порезанными руками".
else:
    show y_sticker at sticker_right # Чиби Юри.
if persistent.playthrough == 2 and chapter == 2:
    show m_sticker at sticker_m_glitch # Чиби Моники.
show l_sticker at sticker_mid
```

b.

8. Определите свои стикеры и анимации под комментарием определения изображений.
9. Сохраните изменения и запустите DDLC. Если всё хорошо, игра запустится, и ваш персонаж в мини-игре будет отображаться и вести себя ожидаемым образом.

Авторы

Спасибо Elckarow за отзывы и основную идею для этой переделки.